Линейная алгебра в Julia

Лабораторная работа №4

Дагделен Зейнап Реджеповна

2026-10-22

Содержание I

1. Информация
2. Введение
3. Теоретическая часть
4. Прикладные задачи
5. Заключение

Раздел 1

Информация

Докладчик

- Дагделен Зейнап Реджеповна
- студентка НКНбд-01-23
- Российский университет дружбы народов им. П. Лумумбы
- 1132236052@rudn.ru



Раздел 2

Введение

Актуальность

- Линейная алгебра — фундамент машинного обучения, физики, экономики.
- Эффективность матричных вычислений критически важна.
- Язык **Julia** сочетает скорость C и удобство Python.
- Специализированные пакеты позволяют избежать ручной реализации алгоритмов.

Цели и задачи

- **Цель:** Изучить возможности пакетов Julia для линейной алгебры и оценить эффективность операций.
- **Задачи:**
  - Освоить базовые операции с матрицами и векторами.
  - Изучить факторизации (LU, QR) и спектральное разложение.
  - Провести бенчмаркинг вычислений.
  - Решить прикладные задачи (СЛАУ, модель Леонтьева).

Раздел 3

Теоретическая часть

Поэлементные операции

- Работают независимо с каждым элементом массива.
- Функции `sum`, `prod`, `mean` в пакете `Statistics`.
- Векторизация операций (`.` — броадкастинг) ускоряет вычисления.
- Позволяет избегать циклов `for`.

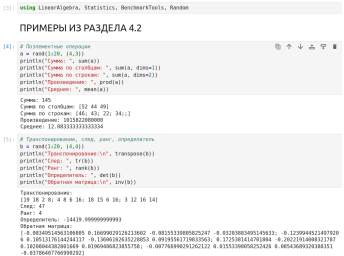


Рисунок 1: Поэлементные операции и операции над матрицами

Нормы и расстояния

- **Евклидова норма:** $\|\vec{X}\|_2 = \sqrt{\sum x_i^2}$
- **p-норма:** обобщение евклидовой нормы.
- **Расстояние** между векторами: $\|\vec{X} - \vec{Y}\|_2$.
- **Угол** между векторами через скалярное произведение.

```
[6]: # Нормы и расстояния
X = [2, 4, -5]
Y = [1, -1, 3]
println("Норма X (евклидова): ", norm(X))
println("p-норма (p=1): ", norm(X, 1))
println("Расстояние между X и Y: ", norm(X - Y))
println("Угол между векторами (рад): ", acos((X' * Y) / (norm(X) * norm(Y))))
```

```
Норма X (евклидова): 6.708203932499369
p-норма (p=1): 11.0
Расстояние между X и Y: 9.486832980505138
Угол между векторами (рад): 2.4404387889469252
```

```
[8]: #Умножение матриц, скалярное произведение,
     #единичная
A = rand(1:10, (2,3)); B = rand(1:10, (3,4))
println("A * B =\n", A * B)
I3 = Matrix{Int}(I, 3, 3) # Единичная матрица
println("Единичная 3x3:\n", I3)
println("Скалярное произведение dot: ", dot(X, Y))
println("Скалярное произведение X'Y: ", X'Y)
```

```
A * B =
[157 160 126 123; 154 136 105 90]
Единичная 3x3:
[1 0 0; 0 1 0; 0 0 1]
Скалярное произведение dot: -17
Скалярное произведение X'Y: -17
```

Рисунок 2: Нормы, расстояния и умножение матриц

Факторизации матриц

- **LU-разложение:** $A = LU$ (нижнетреугольная и верхнетреугольная).
- **QR-разложение:** $A = QR$ (ортогональная и верхнетреугольная).
- Применяются для решения СЛАУ и поиска собственных значений.
- В Julia реализованы в пакете `LinearAlgebra`.



Рисунок 3: Факторизации и спектральное разложение

Специальные структуры данных

- Julia поддерживает явное указание типа матрицы: `Symmetric`, `Diagonal`, `Tridiagonal`.
- Указание структуры позволяет алгоритмам **игнорировать проверки** и работать быстрее.
- «Зашумленная» матрица теряет свойства симметрии, что резко замедляет вычисления.

```
[10]: # BenchmarkTools (спец. структуры)
n = 1000
A = randn(n,n)
Asym = A + A'
Asym_noisy = copy(Asym)
Asym_noisy[1,2] += 5eps()
Asym_explicit = Symmetric(Asym_noisy)

# Сравнение скорости
println("Обычная симметричная:")
@btime eigvals(Asym);
println("Зашумленная:")
@btime eigvals(Asym_noisy);
println("Явно указанная Symmetric:")
@btime eigvals(Asym_explicit);
```

```
Обычная симметричная:
  128.718 ms (18 allocations: 7.98 MiB)
Зашумленная:
  724.310 ms (27 allocations: 7.93 MiB)
Явно указанная Symmetric:
  125.327 ms (18 allocations: 7.98 MiB)
```

Рисунок 4: Оценка эффективности (BenchmarkTools)

Раздел 4

Прикладные задачи

СЛАУ и псевдообратная матрица

- Для квадратных систем используется оператор \.
- Для переопределенных систем — pinv (псевдообратная матрица).
- Проверка совместности через ранги матриц (rank).

```
[13]: println("\n--- С трами неизвестными ---")
      # a) x+y+z=2; x-y-2z=3
      println("a) ", pinv([1 1 1; 1 -1 -2]) * [2, 3])
      # b) x+y+z=2; 2x+2y-2z=4; 3x+y+z=1
      println("b) ", [1 1 1; 2 2 -2; 3 1 1] \ [2, 4, 1])
      # c) x+y+z=1; x+y+2z=0; 2x+2y+3z=1
      println("c) Ранг A: ", rank([1 1 1; 1 1 2; 2 2 3]), ", Ранг A|b: ", rank([1 1 1 1; 1 1 2 0; 2 2 3 1]))
      # d) x+y+z=1; x+y+2z=0; 2x+2y+3z=0
      println("d) Ранг A: ", rank([1 1 1; 1 1 2; 2 2 3]), ", Ранг A|b: ", rank([1 1 1 1; 1 1 2 0; 2 2 3 0]))
```

```
--- С трами неизвестными ---
a) [2.2142857142857144, 0.3571428571428576, -0.5714285714285716]
b) [-0.5, 2.5, 0.0]
c) Ранг A: 2, Ранг A|b: 2
d) Ранг A: 2, Ранг A|b: 3
```

4.4.3. Операции с матрицами (Диагонализация, Степени и Собственные значения)

```
[14]: # 1. Приведение матриц к диагональному виду
      # a)
      A_a = [1 -2; -2 1]
      eig_a = eigen(A_a)
      println("1a) Диагональная матрица Λ:\n", Diagonal(eig_a.values))
      println("    Матрица собственных векторов V:\n", eig_a.vectors)
```

```
1a) Диагональная матрица Λ:
Diagonal([-1.0, 3.0])
    Матрица собственных векторов V:
[-0.7071067811865475 -0.7071067811865475; -0.7071067811865475 0.7071067811865475]
```

```
[15]: # b)
      A_b = [1 -2; -2 3]
      eig_b = eigen(A_b)
      println("\n1b) Диагональная матрица Λ:\n", Diagonal(eig_b.values))
      println("    Матрица собственных векторов V:\n", eig_b.vectors)
```

```
1b) Диагональная матрица Λ:
Diagonal([-0.2360679774997897, 4.236067977499979])
    Матрица собственных векторов V:
[-0.8506508083520399 -0.5257311121191335; -0.5257311121191335 0.8506508083520399]
```

Рисунок 5: Системы линейных уравнений с двумя неизвестными

Диагонализация и спектр

- `eigen(A)` возвращает собственные значения и векторы.
- Диагональная матрица Λ строится через `Diagonal`.
- Спектральное разложение: $A = V\Lambda V^{-1}$.

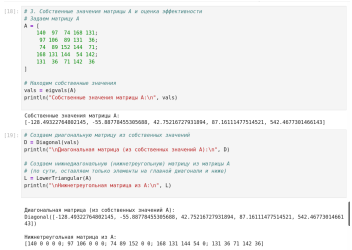


Рисунок 6: Приведение матриц к диагональному виду

Степени и корни матриц

- Возведение в степень (^) и извлечение корней (`sqrt`) работают для матриц.
- Корни могут давать **комплексные числа**, если матрица не имеет вещественного спектра.
- Это связано с комплексными собственными значениями.



Рисунок 7: Вычисления (степени и корни)

Модель Леонтьева

- Уравнение: $x - Ax = y$ (межотраслевой баланс).
- Матрица A **продуктивна**, если решение $x \geq 0$ при любом $y \geq 0$.
- **Критерий 2:** все элементы $(E - A)^{-1} \geq 0$.
- **Критерий 3 (спектральный):** $\max |\lambda| < 1$.

```
--- 3d. Матрица 3x3 ---
Матрица 3d:
  Собственные значения: [0.02679491924311228, 0.1, 0.37320508075688774]
  Критерий 2 (матрица (E-A)⁻¹ >= 0): true
  Критерий 3 (спектральный max|λ| < 1): true (max|λ| = 0.373)
```

Рисунок 8: Проверка продуктивности матриц по критериям

Раздел 5

Заключение

Главный вывод

Использование специализированных структур данных и встроенных функций Julia позволяет не только упростить код, но и **значительно ускорить** вычисления. Явное указание типа матрицы (например, `Symmetric`) дает выигрыш в скорости в разы по сравнению с обычными массивами.